

CMAN — Arithmetic Intensity of `compute_core`

$SV = \text{support vector}$

1. Subfunctions

The `compute_core` function consists of a **dist** subfunction and a **horner** subfunction.

- **dist** — inputs: two 256-element $\mathbb{Q}_{6.10}$ feature vectors; output: 32-bit integer d^2
 - **horner** — input: $-\gamma \cdot d^2$ (re-expressed as $\mathbb{Q}_{6.10}$); output: $K \in [0, 1]$, also represented in $\mathbb{Q}_{6.10}$ for consistency
-

2. Operation Counts

Since $\mathbb{Q}_{6.10}$ fixed-point is used throughout, throughput is measured in **OPs** rather than FLOPs.

Distance function:

$$d^2 = \sum_{i=0}^{255} (x_i - sv_i)^2 \quad \text{per SV}$$

Step	Count
Subtractions	256
Multiplications	256
Accumulations	256
Total per SV	$3 \times 256 = 768$ OPs

With 500 SVs: $768 \times 500 = 384,000$ OPs/beat

Horner function — 3 products + 3 additions per SV:

$$6 \times 500 = 3,000 \text{ OPs/beat}$$

Total OPs per beat:

$$384,000 + 3,000 = 387,000 \text{ OPs/beat}$$

3. Memory Traffic

dist — **no reuse**. Each SV must be loaded from SRAM: 256 features $\times$ 2 bytes $\times$ 500 SVs:

$$256 \times 2 \times 500 = 256,000 \text{ bytes}$$

Input feature vector **x**: $256 \times 2 = 512$ bytes, loaded once.

Total without reuse: $256,000 + 512 \approx 256,512$ bytes $\approx 2 \times 128,000$ bytes

horner — **LUT reuse**. The LUT is a 256-entry $\mathbb{Q}_{6.10}$ vector = 512 bytes, loaded **once**. Without reuse it would require $512 \times 500 = 256,000$ additional bytes per beat. Loaded once, LUT traffic is negligible relative to the SV traffic.

4. Arithmetic Intensity

$$AI = \frac{\text{OPs}}{\text{bytes transferred}}$$

No reuse (LUT reloaded per SV):

$$AI = \frac{3 \times 128,000}{4 \times 128,000} = \frac{3}{4} = 0.75 \text{ OPs/byte}$$

LUT preloaded (one-time load):

$$AI = \frac{3 \times 128,000}{2 \times 128,000} = \frac{3}{2} = 1.5 \text{ OPs/byte}$$

5. Design Implications

The design is **severely memory-bound** ($AI \approx 1.5$ OPs/byte). It relies on a 40 MHz clock to minimize average power consumption. To achieve better throughput, the design would require:

- Higher memory bandwidth
- A **batch configuration** for `compute_core` (see *design_summary* Appendix B)

If battery technology improves, the power constraint could be relaxed and a batch algorithm could be deployed in a portable device.